

Практическое занятие 3.2

Первая нейронная сеть на Keras и TensorFlow

Цели работы

1. Научиться строить и обучать простую модель нейронной сети с использованием Keras.
2. Освоить процесс компиляции и обучения модели: *модель* → *оптимизатор* → *функция потерь* → *метрики* → *обучение* → *оценка*.
3. Изучить работу с набором данных **Fashion-MNIST**.
4. Провести исследование влияния гиперпараметров (архитектуры, шага обучения, размера батча, регуляризации) на результаты модели.

Теоретическое введение

Keras — высокоуровневая библиотека глубокого обучения, встроенная в TensorFlow 2.x. Она предоставляет удобный интерфейс для построения, компиляции, обучения и оценки нейросетевых моделей.

Модель в Keras может быть создана:

- через **Sequential API** — последовательное соединение слоёв;
- через **Functional API** — для более сложных графов.

Основные этапы обучения:

1. Подготовка данных (нормализация, разделение train/val/test).
2. Определение архитектуры модели.
3. Компиляция (optimizer + loss + metrics).
4. Обучение (fit).
5. Оценка (evaluate) и визуализация (history).
6. Сохранение модели и дальнейшие эксперименты.

1. Импорт библиотек и загрузка данных Fashion-MNIST

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

print("TensorFlow version:", tf.__version__)
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.fashion_mnist.load_data()
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]

print("Train shape:", x_train.shape, "Test shape:", x_test.shape)
```

Комментарий: Данные состоят из изображений 28×28 пикселей в градациях серого и меток (0–9) — тип.

2. Подготовка данных

Нормализация

```
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32") / 255.0
```

Разделение на валидацию

```
val_count = 5000
x_val, y_val = x_train[-val_count:], y_train[-val_count:]
x_train, y_train = x_train[:-val_count], y_train[:-val_count]
```

Проверка входных данных

```
plt.imshow(x_train[0], cmap="gray")
plt.title(f"Класс: {class_names[y_train[0]]}")
plt.axis("off"); plt.show()
```

Зачем нормализовать?

Чтобы ускорить обучение и предотвратить взрыв градиентов — значения пикселей должны быть в интервале $[0, 1]$.

3. Построение модели Keras Sequential

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model.summary()
```

Пояснения

Слой	Назначение
Flatten	Преобразует матрицу 28×28 в вектор 784 элементов
Dense(128, ReLU)	Скрытый слой из 128 нейронов с нелинейностью ReLU
Dense(10, Softmax)	Выходной слой 10 классов, Softmax даёт вероятности

Количество параметров: $(784 \times 128 + 128) + (128 \times 10 + 10) \approx 101\,770$ весов.

4. Компиляция и обучение модели

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
```

```

x_train, y_train,
epochs=10,
batch_size=128,
validation_data=(x_val, y_val),
verbose=2
)

```

Комментарии

- **optimizer** — способ обновления весов (Adam, SGD и др.);
- **loss** — функция потерь (для многоклассовой классификации — `categorical_crossentropy` или `sparse_categorical_crossentropy`);
- **metrics** — метрики для оценки качества (accuracy — доля верных предсказаний).

5. Анализ и визуализация обучения

```
hist = history.history
```

```

plt.figure(figsize=(6,4))
plt.plot(hist["loss"], label="train_loss")
plt.plot(hist["val_loss"], label="val_loss")
plt.xlabel("Эпоха"); plt.ylabel("Loss"); plt.legend(); plt.show()

```

```

plt.figure(figsize=(6,4))
plt.plot(hist["accuracy"], label="train_acc")
plt.plot(hist["val_accuracy"], label="val_acc")
plt.xlabel("Эпоха"); plt.ylabel("Accuracy"); plt.legend(); plt.show()

```

Интерпретация:

- Если `val_loss` начинает расти при падении `train_loss` → переобучение.
- Если кривые стабилизируются и идут вместе → обучение корректное.

6. Оценка качества и матрица ошибок

```

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Тестовая точность: {test_acc:.4f}")
y_pred = model.predict(x_test, verbose=0).argmax(axis=1)
cm = confusion_matrix(y_test, y_pred, labels=list(range(10)))
disp = ConfusionMatrixDisplay(cm, display_labels=class_names)
disp.plot(xticks_rotation=45, cmap="Blues"); plt.show()

```

Интерпретация: проверьте, какие классы чаще всего путаются (обычно «Shirt» и «T-shirt»).

7. Улучшение модели: регуляризация и Dropout

```

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

reg_model = models.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(256, activation="relu",
kernel_regularizer=tf.keras.regularizers.l2(1e-4)),
    layers.Dropout(0.3),
    layers.Dense(128, activation="relu",
kernel_regularizer=tf.keras.regularizers.l2(1e-4)),

```

```

        layers.Dropout(0.3),
        layers.Dense(10, activation="softmax")
    ])

reg_model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

es = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)
mc = ModelCheckpoint("best_fmnist.keras", monitor="val_loss",
                    save_best_only=True)

reg_hist = reg_model.fit(x_train, y_train,
                        epochs=20, batch_size=128,
                        validation_data=(x_val, y_val),
                        callbacks=[es, mc],
                        verbose=2)

```

Пояснения

- **Dropout(0.3)** — зануляет 30 % нейронов во время обучения.
- **L2-регуляризация** — штрафует большие веса → предотвращает переобучение.
- **EarlyStopping** — автоматически останавливает обучение при росте val_loss.
- **ModelCheckpoint** — сохраняет лучшую модель.

8. Сравнение моделей (базовой и регуляризованной)

```

def plot_history(h, title_prefix=""):
    hist = h.history
    fig, ax = plt.subplots(1, 2, figsize=(10, 4))
    ax[0].plot(hist["loss"], label="train"); ax[0].plot(hist["val_loss"],
label="val")
    ax[0].set_title(f"{title_prefix} Loss"); ax[0].legend()
    ax[1].plot(hist["accuracy"], label="train");
ax[1].plot(hist["val_accuracy"], label="val")
    ax[1].set_title(f"{title_prefix} Accuracy"); ax[1].legend()
    plt.show()

plot_history(history, "Base")
plot_history(reg_hist, "Regularized")

```

Выводы: сравните по графикам и тестовой точности, уменьшилось ли переобучение.

9. Мини-исследование (работа в 5 подгруппах)

Подгруппа	Направление исследования	Что сравнить	Что внести в отчёт
A	Оптимизаторы (Adam, SGD, RMSprop)	Скорость сходимости и качество	Графики loss/acc, тестовая точность
B	Глубина сети (1–3 слоя, 128–512 нейронов)	Влияние архитектуры	Таблица результатов
C	Регуляризация (L2 = 1e-3/1e-4, Dropout = 0.2/0.5)	Переобучение и стабильность	Сравнение val_loss
D	Шаг обучения (LR = 1e-2, 1e-3, 1e-4)	Сходимость и устойчивость	Графики loss, тезисы

Подгруппа	Направление исследования	Что сравнить	Что внести в отчёт
Е	Размер батча (64, 128, 256)	Время и точность	Баланс скорость/качество

Каждая группа фиксирует:

- гиперпараметры;
- лучшие значения валидации и теста;
- графики и 3–5 тезисов вывода.

10. Контрольные вопросы

1. Что задаётся в `compile()` и почему это обязательно?
2. Почему для многоклассовой классификации используется `softmax`?
3. Как распознать переобучение по графикам?
4. Как работают `EarlyStopping` и `ModelCheckpoint`?
5. Чем отличаются `Sequential` и `Functional API`?
6. Что изменится, если не нормализовать данные?

Структура отчёта по практической работе

1. Титульный лист (название, Ф.И.О., дата).
2. Цель работы.
3. Теоретические сведения (кратко о Keras, `Sequential` модели).
4. Ход выполнения (код, результаты, графики).
5. Исследовательская часть (описание эксперимента подгруппы).
6. Ответы на контрольные вопросы.
7. Выводы (3–5 пунктов).

Выводы по практике 3.2

- Keras — интуитивный инструмент для создания и обучения нейронных сетей.
- Процесс обучения включает четыре ключевых шага: определение модели, компиляция, обучение, оценка.
- Регуляризация и Dropout предотвращают переобучение и повышают обобщающую способность.
- Настройка гиперпараметров (`learning rate`, `batch size`, архитектура) позволяет повысить точность и стабильность результатов.
- Полученные навыки будут использоваться в дальнейших темах (сверточные сети, рекуррентные архитектуры, оптимизация нейросетей).